Data Analyst — Technical Case

Welcome. The goal of this exercise is to give us — and you — a realistic sense of how you approach a messy, real-world data problem end-to-end.

You will receive five files: three CSVs of raw flight-operations data, one per airline (`ak_*`, `jt_*`, `y4_*`); one airports master CSV (`airports_master.csv`); and one ticket prices CSV (`ticket_prices.csv`). Schemas across the three operational files are not the same. Full documentation for every dataset is in `Annex_Dataset_Documentation.pdf` — read it before you start writing SQL.

What we are evaluating

- **SQL fluency** — readable, correct, performant queries; sensible CTEs, window functions, joins.
- **Data quality and QA instincts** — finding, documenting, and resolving issues in raw data without being told what to look for.
- **Reasoning under ambiguity** — the assumption you made, why, and what you'd do differently with more time.
- **Attention to detail** — precise outputs, precise formatting, precise communication.
- **Following detailed specifications** — particularly in the dashboard section, where the spec is intentionally prescriptive.
- **Written communication** — your `findings.pdf` and inline SQL comments should let someone else on the team pick up your work in fifteen minutes.
- **Analytical curiosity** — one part is open-ended. We want to see what you find interesting and why.

A note on AI tools: you may use them and we expect you to. The final code, output, and reasoning have to be yours and you have to be able to defend every choice — we will ask you to walk through your work in the follow-up interview.

Setup

Use any SQL engine — Postgres, BigQuery, Snowflake, DuckDB, SQLite, SQL Server. Tell us which one, because some cleaning steps (especially the JSON columns in the Lion Air file) have dialect-specific syntax. Mention any non-standard extensions you use. If your dialect doesn't support JSON natively (SQLite, older SQL Server), expect to pre-process the JT file in Python or pandas before loading.

Load the CSVs however you like — `COPY`, `LOAD DATA`, `read_csv_auto`, or a small ingestion script. Include any loader script in your deliverable.

For the dashboard, use **Looker Studio, Power BI, or Tableau Public** (in that order of preference). Do not use Excel or Google Sheets. Submit a public/shareable link plus a screenshot of the page.

Time budget

This is the realistic time we expect a strong submission to take. It is **expected time, not maximum time** — if you are running over, prioritise the required parts and write down what you would have done.

Section	Expected time
Part 1 — Cleaning + findings write-up	3–4 hours
Part 2 — Q2 (daily counts)	~30 minutes
Part 2 — Q3 (problematic flights)	1.5–2 hours
Part 3 — Dashboard (required page)	2–3 hours
Total — required scope	**7–9 hours**
Q4 + optional bonus dashboard page	+2–3 hours if attempted

We would rather see a well-scoped, well-explained, slightly smaller submission than a sprawling one.

Part 1 — Data Cleaning (required)

Produce a single, unified, clean table of flight operations across the three airlines, suitable for the analytical questions that follow. Document everything you found and everything you did.

Deliverables for this part

1. `findings.pdf` — a short write-up (1–3 pages) covering:

- **Data quality findings**: for each issue you identified, a short paragraph explaining what it is, where it appears (file, column, roughly how many rows), why it matters, and how you decided to resolve it (drop, fix, flag, ignore — and your reasoning).
- **Conclusions for Q2 and Q3**: for each required analytical question, a brief paragraph (3–6 sentences) explaining what the result tells us and any caveats worth flagging.

2. `q1_cleaning_merging.sql` — a SQL script that, when run against the three raw operational CSVs, produces a single unified table called `flights_unified` with at least the following columns:

Column	Type	Notes

`flight_uid`	string	Surrogate primary key, unique per cleaned row.
`airline_iata`	string(2)	`AK`, `JT`, or `Y4`.
`airline_icao`	string(3)	`AXM`, `LNI`, or `VOI`.
`flight_number`	string	Numeric portion only, no whitespace, no quotes.
`departure_date`	date	Operational date of the flight.
`dep_airport_iata`	string(3)	Validated 3-letter uppercase IATA.
`arr_airport_iata`	string(3)	Validated 3-letter uppercase IATA.
`scheduled_departure`	timestamp	Local time at origin.
`estimated_departure`	timestamp	Local time at origin. May be null.
`actual_departure`	timestamp	Local time at origin. May be null.
`scheduled_arrival`	timestamp	Local time at destination.
`estimated_arrival`	timestamp	Local time at destination. May be null.
`actual_arrival`	timestamp	Local time at destination. May be null.
`flight_status`	string	A small, controlled vocabulary that you define and document.
`dep_delay_minutes`	integer	`actual_departure − scheduled_departure` in minutes (signed). May be null.
`arr_delay_minutes`	integer	`actual_arrival − scheduled_arrival` in minutes (signed). May be null.
`aircraft_type`	string	Best-available value, may be null.

The script should be re-runnable end-to-end without manual edits.

Things to think about (not exhaustive — finding the rest is the test)

- The three files do not share a common schema. Map each one to the unified shape.
- One file embeds key information inside JSON-like columns. Parse them.
- Some identifier fields contain stale or incorrect substrings that must not be used as the source of truth — pick the right column for each piece of information.
- Watch for inconsistent casing, whitespace, padding, quoting, spelling variants, and date format variants.
- Watch for exact duplicates and near-duplicates.
- Watch for impossible rows (impossible airport codes, impossible date relationships, impossible routes).
- Watch for literal placeholder strings that look like values but aren't.
- Watch for status vocabularies that vary across and within files.
- Decide and document how you handle rows that are entirely empty in their key fields.

We are not looking for you to find every issue — we are looking for a thoughtful, systematic approach and clean documentation of what you did and why.

Part 2 — Analytical Questions

Questions 2 and 3 are required. **Question 4 is optional** — see the note at the top of that section. All questions are answered against the unified table from Part 1, optionally joined to `airports_master` and `ticket_prices`.

Question 2 (Easy, required) — Daily flight counts per airline

Write a SQL query that returns the number of flights per airline per day across the full date range present in your unified dataset.

Output columns: `flight_date` (date), `airline_iata` (string), `flight_count` (integer). Sort by `flight_date` ascending, then `airline_iata` ascending.

Save as `q2.sql`. Include the first 20 rows of the result as a comment at the bottom of the file.

Question 3 (Medium, required) — Top problematic flights per route per month

For each `(dep_airport_iata, arr_airport_iata)` route and each calendar month, identify the top 3 most problematic flight numbers.

A flight is **problematic** if either:

- the difference between `estimated_departure` and `scheduled_departure` is greater than 30 minutes, or
- the difference between `estimated_arrival` and `scheduled_arrival` is greater than 30 minutes.

Pre-step (deduplication): if there are multiple records for the same `flight_number` on the same `departure_date`, keep only one record per `(flight_number, departure_date)` — the record with the worst condition in this priority order: (1) largest departure delay (estimated − scheduled), then (2) largest arrival delay (estimated − scheduled). Only after this dedup do

you compute monthly aggregates.

For each `(route, month)`, compute the total number of flights, the number of problematic flights, the percentage of problematic flights (rounded to two decimals), and a ranking of flight numbers within that `(route, month)` by: (1) number of times the flight number was problematic (descending), then (2) average departure delay (descending), then (3) average arrival delay (descending).

Return only the top 3 flight numbers per `(route, month)`, with these columns: `dep_airport_iata`, `arr_airport_iata`, `month` (format `YYYY-MM`), `total_flights`, `problematic_flights`, `problematic_pct`, `flight_number`, `times_problematic`, `avg_dep_delay_min`, `avg_arr_delay_min`, `rank_in_route_month` (1, 2, or 3).

Order the result by `dep_airport_iata`, `arr_airport_iata`, `month`, `rank_in_route_month`. Save as `q3.sql`.

Question 4 (Hard, optional) — Estimated revenue from each airline's 5th-busiest destination

This question is optional. If you'd rather invest your time in deeper data-quality analysis, a more polished dashboard, or a richer Part 3 investigation, you can skip Q4 with no penalty. If you do attempt it, we'll weight it as a meaningful positive — but we'd rather see strong work on the required parts than a rushed Q4. Your call.

For each airline, identify the 5th-busiest destination airport by passenger movement in the analysis window, and estimate the revenue that destination contributed to that airline over the same window.

Use the unified flights table (frequencies), `airports_master` (for joining and reference), and `ticket_prices` (for fares). Make and document any assumptions you need to fill in missing information — in particular: aircraft seat capacity (operational data only gives type/family, not seat count); load factor / occupancy (not provided); which fare to apply per flight (cabin mix is not given); how to handle routes with no published fare; how to handle the `airline_iata` ↔ `airline_icao` mapping when joining to prices.

Return a single table with columns: `airline_iata`, `destination_iata`, `destination_name`, `passenger_movement_rank` (should be 5 for every row), `flights_count`, `estimated_passengers`, `estimated_revenue_usd`. Save as `q4.sql`.

In `q4_analysis.pdf`, write 1–2 pages explaining: which assumptions you made and where each value came from (cite sources); what you'd need from a stakeholder to make this estimate properly; which part of your answer you're least confident about and why; a sensitivity check showing how much the answer changes if your load-factor assumption is ±10 percentage points off.

Part 3 — Dashboard

Build a dashboard in Looker Studio, Power BI, or Tableau Public (no Excel, no Google Sheets) that presents the operational story of these three airlines.

The dashboard must contain **one required page** (described below). One **optional bonus page** is described after it. We are deliberately being prescriptive about formatting because we want to evaluate how carefully you read and follow specifications. Most visualisations on the required page reuse work from Parts 1 and 2 — the dashboard is presentation work, not new analysis.

Submit as `dashboard.pdf`: a single PDF with the public/shareable link to your live dashboard on its first page, followed by a full screenshot of the required page. If you also build the optional bonus page, package it as a separate PDF (`pattern_you_found.pdf`).

Global formatting rules (apply to both pages)

Element	Specification
Page size	1280 × 720 px (16:9). Portrait is not acceptable.
Background	`#F4F6F8` (light grey).
Title bar	Top of every page, height 64 px, background `#0B2545` (dark navy). Page title in white, 22pt, font Inter (or Arial if unavailable), left-aligned, 24px left padding.
Subtitle row	Directly below title bar, height 40 px, background `#FFFFFF`. Subtitle in `#333333`, 12pt italic, left-aligned, 24px left padding. Each page has a different subtitle (specified below).
Airline colors	`AK` = `#E63946`, `JT` = `#1D4E89`, `Y4` = `#7B2CBF`. For "Other" / "All", use `#6C757D`. Use these consistently wherever an airline appears.
Counts	Thousands separators, no decimals (e.g. `1,234`).
Percentages	One decimal with `%` suffix (e.g. `12.4%`).
Currency (USD)	`$` prefix, thousands separators, no decimals (e.g. `$12,345`).
Distances (km)	Thousands separators, no decimals, `km` suffix (e.g. `1,234 km`).
Dates	`DD-MMM-YYYY` (e.g. `01-Jun-2025`). Months in chart axes may be abbreviated as `Jun`, `Jul`, `Aug`.
Chart titles	14pt bold, color `#0B2545`, left-aligned above each chart, 8px vertical spacing to chart body.
Footer	Bottom of every page, height 24px, background `#FFFFFF`. Text:

	`Source: internal operations extract, Jun-Aug 2025. Prepared by <your full name>.` in 9pt italic, color `#666666`, right-aligned, 24px right padding.
Filters / slicers	At least one date-range and one airline filter on every page, top-right, below the title bar and above the chart area.

Required page — Airline Operations Overview

Page title: `Airline Operations Overview`. **Subtitle:** `Operational performance and the most problematic routes, June–August 2025.`

This page must contain at minimum the four elements below. All are sourced from work you've already done in Parts 1, 2, and (optionally) 4.

a. KPI strip — Top of the page, just below the subtitle row, height 96 px. Three KPI cards (or four if you attempted Q4) laid out left-to-right with equal width:

1. Total flights (count, with a comparison vs the period midpoint as a small grey delta below the number).
2. Average departure delay (minutes, one decimal, suffix `min`).
3. On-time percentage (defined as `dep_delay_minutes <= 15 AND not cancelled`, percentage with one decimal).
4. Total estimated revenue (USD) — only if you attempted Q4. If you skipped Q4, do not include this card; keep the strip at three cards.

Each card has a 1-pixel border in `#DDDDDD`, white background, metric label in 10pt medium grey, and metric value in 24pt bold in `#6C757D`.

b. Stacked bar chart — Daily flight count by airline — Renders the result of your Q2 query. X-axis = date; y-axis = flight count; stacked by airline using the airline palette. Y-axis must start at zero. Include data labels only on the top-most segment of each bar.

c. Donut chart — Flight status distribution — Across all three airlines combined, using the controlled status vocabulary from Part 1. Donut hole diameter = 50% of chart diameter. Show percentages inside segments only when the segment is ≥ 5%.

d. Top problematic routes table — Sourced directly from your Q3 result. Show the top 10 `(route, month)` combinations by `problematic_pct`, with these columns:

Column	Notes
Route	Format: `DEP → ARR` (e.g. `KUL → SIN`).
Month	Format: `MMM-YYYY` (e.g. `Jun-2025`).
Total flights	From Q3.
Problematic flights	From Q3.
% problematic	From Q3, with the color scale below.

The `% problematic` column must use a color scale: green at 0%, yellow at 25%, red at 50% or above, linearly interpolated. Sort by `% problematic` descending, then by total flights descending as a tiebreaker.

Optional bonus page — A Pattern You Found

Page title: `A Pattern You Found`. **Subtitle:** `<A short title (max 12 words) describing the pattern you found.>`

This optional bonus is your chance to surface something interesting in the data on your own. We are explicitly not telling you what to look for — the point is to see what catches your attention.

If you choose to attempt it, build at least one visualisation that highlights a pattern, anomaly, or operational risk you consider commercially or operationally meaningful. Alongside the visualisation(s), include a text box (positioned so it doesn't overlap the visuals) with a one-sentence headline (bold, 14pt, color `#0B2545`) and a paragraph (50–120 words) explaining what the pattern is, why it matters, and what you'd recommend doing about it (or what additional data you'd request).

Package this bonus page as `pattern_you_found.pdf` — a single PDF containing both the screenshot and the written justification.

Submission

Zip your full deliverable (excluding the raw CSVs) and send it to the email address you received this case from.

If something in this brief is unclear or ambiguous, make a reasonable assumption, document it in your `findings.pdf`, and proceed. The ability to do that — rather than blocking on clarification — is part of what we're looking for.

Deliverables checklist

Use these exact file names — they make our review faster and more consistent across candidates.

Required:

- `q1_cleaning_merging.sql` — re-runnable script that loads the raw CSVs, cleans them, and produces `flights_unified`.
- `q2.sql` — Question 2 query.
- `q3.sql` — Question 3 query.

- `findings.pdf` — data quality findings, fixes, and brief conclusions for Q2 and Q3.
- `dashboard.pdf` — public/shareable link plus a full screenshot of the required page.

Optional (only if attempted):

- `q4.sql` — Question 4 query.
- `q4_analysis.pdf` — Question 4 reasoning, assumptions, and sensitivity check.
- `pattern_you_found.pdf` — combined screenshot and written justification of the optional open-investigation pattern.

Do not include: the raw CSV files we sent you, database dump files, `.duckdb`/`.sqlite` files, virtual environments, or any other large binary artifacts.

Good luck. We're looking forward to reading your work.